PRODUCT REQUIREMENTS DOCUMENT

Vision-Language Model for Manufacturing Defect Detection

LLaVA-Based Quality Control System

Project Name	VLM Defect Detection System
Version	1.0
Date	January 26, 2026
Author	Pranav Koduru
Status	In Development

Table of Contents

1. Executive Summary
2. Problem Statement
3. Goals & Objectives
4. Target Users & Stakeholders
5. Functional Requirements
6. Technical Requirements
7. Success Metrics & KPIs
8. Technical Architecture
9. Implementation Plan
10. Risk Assessment & Mitigation
11. Project Timeline & Milestones
12. Deployment Strategy
13. Future Enhancements
14. Appendices

1. Executive Summary

This document outlines the requirements for developing a Vision-Language Model (VLM) based quality control system for manufacturing defect detection. The system leverages state-of-the-art multimodal AI technology (LLaVA-1.5-7B) fine-tuned on the MVTec Anomaly Detection dataset to provide intelligent, natural language-based defect analysis.

1.1 Project Overview

The VLM Defect Detection System is an AI-powered quality control tool that combines computer vision with natural language understanding to identify and describe manufacturing defects. Unlike traditional computer vision systems that only classify defects, this system can:

- Understand and respond to natural language queries about product quality
- Describe defects in human-readable language
- Handle zero-shot detection of unseen defect types
- Provide contextual analysis beyond binary pass/fail decisions

1.2 Key Benefits

- 89% accuracy on defect detection across 15 product categories
- Natural language interface reduces training time for quality control operators
- Flexible deployment via cloud API or edge devices
- Cost-effective solution using fine-tuned open-source models
- Demonstrates production-ready ML engineering capabilities for portfolio

1.3 Success Criteria

The project will be considered successful when:

- Model achieves ≥85% accuracy on MVTec AD validation set
- Inference latency <2.5 seconds per image on standard GPU
- Live demo deployed on Hugging Face Spaces with public access
- Complete documentation and codebase published on GitHub
- Project showcased in portfolio and resume for job applications

2. Problem Statement

2.1 Current Challenges

Traditional manufacturing quality control systems face several limitations:

- Manual inspection is time-consuming, expensive, and prone to human error
- Traditional CV systems require extensive labeled data for each defect type
- Binary pass/fail systems lack contextual understanding of defect severity
- Inflexible systems cannot adapt to new defect types without retraining
- No natural language interface for operators to query system capabilities

2.2 Market Opportunity

The industrial quality control market presents significant opportunities:

- Global industrial inspection market: $9.7B (2024) → $15.8B by 2030
- AI-powered quality control adoption growing at 24% CAGR
- Manufacturing labor shortage driving automation demand
- Vision-Language Models emerging as breakthrough technology for 2024-2026
- Limited commercial VLM solutions available for manufacturing

2.3 Target Problem Space

This project specifically addresses defect detection in industrial manufacturing across 15 product categories including:

- Metal components (screws, nuts, metal surfaces)
- Textiles (carpet, leather, grid patterns)
- Plastic products (bottles, capsules, pills)
- Electronic components (transistors, cables)
- Wood and tile surfaces

3. Goals & Objectives

3.1 Primary Goals

1. Demonstrate cutting-edge ML engineering capabilities through Vision-Language Model fine-tuning
2. Build a production-ready defect detection system suitable for real-world deployment
3. Create a compelling portfolio project for AI/ML Engineer job applications
4. Showcase end-to-end ML project execution from data preparation to deployment

3.2 Technical Objectives

- Fine-tune LLaVA-1.5-7B model using LoRA for parameter-efficient training
- Achieve minimum 85% accuracy on zero-shot defect classification
- Optimize inference to <2.5s latency on T4 GPU
- Generate 3,000+ training samples from MVTec AD dataset
- Implement complete MLOps pipeline: training, evaluation, deployment
- Create interactive demo with natural language query interface

3.3 Business Objectives

- Complete project within 3-4 weeks (20-25 active hours)
- Publish on Hugging Face Hub for public access and credibility
- Achieve GitHub repository visibility through comprehensive documentation
- Leverage project for interview discussions and technical assessments
- Position as unique differentiator among new grad AI/ML candidates

3.4 Learning Objectives

- Master Vision-Language Model fine-tuning techniques
- Gain hands-on experience with LoRA and parameter-efficient fine-tuning
- Learn production ML deployment on cloud platforms
- Develop skills in model evaluation and performance optimization
- Build portfolio demonstrating real-world ML project delivery

4. Target Users & Stakeholders

4.1 Primary Users

Quality Control Operators

- Profile: Manufacturing floor workers responsible for product inspection
- Technical level: Low to moderate (no ML expertise required)
- Needs: Fast, accurate defect identification with minimal training
- Pain points: Fatigue from repetitive inspection, need for consistency
- Value: Natural language interface, visual confirmation of defects

Manufacturing Engineers

- Profile: Process engineers optimizing production lines
- Technical level: High (engineering background, some ML awareness)
- Needs: Detailed defect analytics, integration with existing systems
- Pain points: System downtime for retraining, inflexible legacy systems
- Value: Flexible deployment, API access, detailed defect descriptions

4.2 Secondary Stakeholders

Hiring Managers & Technical Recruiters

- Profile: Decision-makers evaluating AI/ML Engineer candidates
- Technical level: Varies (technical managers to HR recruiters)
- Needs: Evidence of production ML capabilities, end-to-end project execution
- Evaluation criteria: Code quality, documentation, deployed demo, metrics
- Value: Live demo, GitHub repository, comprehensive documentation

ML Research Community

- Profile: Researchers and practitioners exploring VLM applications
- Technical level: Expert (PhD candidates, ML engineers, researchers)
- Needs: Implementation details, reproducible results, code access
- Value: Open-source codebase, training methodology, benchmark results

5. Functional Requirements

5.1 Core Functionality

- **FR-001: Image Upload & Processing -** System shall accept image uploads in PNG, JPG, JPEG formats up to 10MB
- **FR-002: Natural Language Query -** System shall accept text queries like 'Are there any defects?', 'Describe the condition', 'Is this acceptable?'
- **FR-003: Defect Detection -** System shall identify presence/absence of defects with confidence scores
- **FR-004: Defect Description -** System shall generate human-readable descriptions of detected defects
- **FR-005: Multi-Category Support -** System shall support all 15 MVTec AD product categories

5.2 User Interface Requirements

- **FR-101: Web Interface -** Provide browser-based interface accessible via public URL
- **FR-102: Image Preview -** Display uploaded image alongside analysis results
- **FR-103: Response Display -** Show model response with formatting and confidence indicators
- **FR-104: Example Queries -** Provide sample questions users can click to demonstrate capabilities
- **FR-105: Mobile Responsive -** Interface shall be usable on mobile devices (tablet/phone)

5.3 API Requirements

- **FR-201: REST API Endpoint -** Expose /predict endpoint accepting image + text query
- **FR-202: Batch Processing -** Support batch predictions for multiple images
- **FR-203: Response Format -** Return structured JSON with defect status, description, confidence
- **FR-204: Error Handling -** Return descriptive error messages for invalid inputs
- **FR-205: Rate Limiting -** Implement fair-use rate limiting for public API

5.4 Model Capabilities

- **FR-301: Zero-Shot Detection -** Detect defects without explicit training on specific defect type
- **FR-302: Multi-Turn Conversation -** Support follow-up questions about the same image
- **FR-303: Confidence Scoring -** Provide reliability indicators for predictions

- **FR-304: Contextual Understanding -** Understand query intent and respond appropriately

6. Technical Requirements

6.1 Model Architecture

- Base Model: LLaVA-1.5-7B (liuhaotian/llava-v1.5-7b from Hugging Face)
- Vision Encoder: CLIP ViT-L/14@336px (openai/clip-vit-large-patch14-336)
- Language Model: Vicuna-7B (LLaMA-based, instruction-tuned)
- Vision-Language Connector: MLP 2-layer with GELU activation
- Fine-tuning Method: LoRA (Low-Rank Adaptation) with rank=128, alpha=256

6.2 Training Requirements

Parameter	Specification
Training Data	MVTec AD Dataset: 3,000+ good samples, 1,700+ defect samples
Training Method	LoRA fine-tuning (0.5% of model parameters)
Training Hardware	Google Colab T4 GPU (16GB VRAM) or A100 (40GB VRAM)
Training Time	3-5 hours on T4, 2-3 hours on A100
Batch Size	4 per device, gradient accumulation steps: 4 (effective batch: 16)
Learning Rate	2e-4 with cosine schedule, warmup ratio: 0.03
Epochs	3 epochs over full training set
Checkpointing	Save every 100 steps, keep last 3 checkpoints

6.3 Performance Requirements

Metric	Target	Minimum Acceptable
Defect Detection Accuracy	89%	85%
Inference Latency (T4 GPU)	<2.0s	<2.5s
Inference Latency (A100 GPU)	<1.0s	<1.5s
Model Size (LoRA)	~256MB	N/A

Memory Usage (Inference)	<8GB VRAM	<10GB VRAM
Throughput	30 images/min	24 images/min
Precision (Defect Class)	92%	88%
Recall (Defect Class)	85%	80%

6.4 Infrastructure Requirements

Development Environment

- Platform: Google Colab Pro (recommended) or Colab Free Tier
- Python Version: 3.10+
- Key Libraries: PyTorch 2.2.0, Transformers 4.37.2, PEFT (LoRA)
- Storage: Google Drive (25GB+ free space for dataset + checkpoints)
- Version Control: GitHub repository with Git LFS for model weights

Production Environment

- Hosting: Hugging Face Spaces (Gradio SDK)
- Compute: CPU Basic (free tier) or GPU Persistent ($0.60/hour)
- Model Storage: Hugging Face Hub Model Repository
- Web Framework: Gradio 4.16+ for interactive demo
- API Framework: FastAPI (optional for advanced deployment)

6.5 Data Requirements

- Dataset: MVTec Anomaly Detection (MVTec AD) - publicly available
- Size: ~4.5GB compressed, 15 product categories, 5,354 images total
- Training Split: 3,629 normal images for training
- Validation Split: 1,725 images (normal + defective) for testing
- Format: PNG images, 700×700 to 1024×1024 resolution
- Preprocessing: Resize to 336×336, normalize per CLIP specifications

7. Success Metrics & KPIs

7.1 Technical Metrics

Metric	Target	Measurement Method
Overall Accuracy	≥85%	Correct predictions / Total predictions on validation set
Defect Detection Precision	≥88%	True Positives / (True Positives + False Positives)
Defect Detection Recall	≥80%	True Positives / (True Positives + False Negatives)
F1 Score	≥85%	2 × (Precision × Recall) / (Precision + Recall)
Inference Speed	<2.5s	Average time per image on T4 GPU
Model Size	<300MB	LoRA adapter weights (compressed)
Training Loss	<0.5	Final cross-entropy loss on training data

7.2 Portfolio & Career Metrics

Metric	Target	Measurement Method
GitHub Stars	20+	GitHub repository stars within 3 months
Live Demo Visitors	100+	Hugging Face Spaces visitor count
Resume Callbacks	30%	Response rate to job applications featuring this project
Interview Mentions	80%	Percentage of interviews where project is discussed
Portfolio Views	200+	Project page views within 3 months

7.3 Project Delivery Metrics

Metric	Target	Measurement Method
Time to MVP	2 weeks	Working model deployed on Hugging Face
Total Development Time	20-25 hours	Active coding/training time
Documentation Completeness	100%	All sections of README completed
Code Quality	>80%	Pylint score on production code
Test Coverage	>60%	Unit test coverage for core functions

8. Technical Architecture

8.1 System Architecture

The system follows a three-tier architecture:

- Frontend Layer: Gradio web interface for user interaction
- Application Layer: Model inference pipeline with preprocessing/postprocessing
- Model Layer: Fine-tuned LLaVA model with LoRA adapters

8.2 Data Flow

5. User uploads image and enters natural language query via web interface
6. Image preprocessed: resize to 336×336, normalize using CLIP statistics
7. Text query tokenized using LLaMA tokenizer with special image tokens
8. Vision encoder (CLIP) processes image → 576 visual tokens (24×24 patches)
9. MLP projector transforms visual tokens to language model embedding space
10. Vicuna language model processes [visual tokens + text query] → generates response
11. Response decoded and formatted for display in web interface
12. Results cached for follow-up queries on same image

8.3 Training Pipeline

13. Data Preparation: MVTec AD downloaded → converted to LLaVA instruction format
14. Model Initialization: Load pre-trained LLaVA-1.5-7B weights from Hugging Face
15. LoRA Configuration: Inject low-rank adapters into attention layers (rank=128)
16. Fine-tuning: Train only LoRA parameters (freezing base model) for 3 epochs
17. Checkpointing: Save model state every 100 steps to Google Drive
18. Evaluation: Test on validation set, compute accuracy/precision/recall
19. Model Merging: Merge LoRA weights into base model for deployment
20. Upload: Push final model to Hugging Face Hub with model card

8.4 Deployment Architecture

Component	Technology	Purpose
Model Hosting	Hugging Face Hub	Store and version model weights
Inference Server	Hugging Face Spaces	Host model and serve predictions

Web Framework	Gradio	Build interactive user interface
Model Loading	Transformers + PEFT	Load model with LoRA adapters
Image Processing	PIL + Torch	Preprocess uploaded images
Response Generation	Transformers AutoModel	Generate text responses

9. Implementation Plan

9.1 Phase 1: Setup & Data Preparation (Days 1-3)

Task	Duration	Deliverable
Set up Google Colab environment	0.5 hour	Working Colab notebook with GPU access
Download MVTec AD dataset	1 hour	4.5GB dataset in Google Drive
Create data conversion script	2 hours	Python script to convert to LLaVA format
Generate training/validation JSON	1 hour	mvtec_train.json (3K samples), mvtec_val.json (1.7K samples)
Validate data format	0.5 hour	Sample verification, statistics report

9.2 Phase 2: Model Fine-Tuning (Days 4-6)

Task	Duration	Deliverable
Clone and install LLaVA	1 hour	LLaVA repository + dependencies installed
Configure training parameters	0.5 hour	Training config with LoRA settings
Start fine-tuning	4 hours	Model training (3-5 hrs on T4 GPU)
Monitor training progress	0.5 hour	Loss curves, checkpoint validation
Evaluate final model	1 hour	Accuracy/precision/recall on validation set

9.3 Phase 3: Model Preparation (Days 7-8)

Task	Duration	Deliverable
Merge LoRA weights	0.5 hour	Single merged model (~14GB)

Test inference locally	1 hour	Verify predictions on sample images
Create model card	1 hour	Documentation with usage examples
Upload to Hugging Face	1 hour	Model published on HF Hub
Verify model download	0.5 hour	Test loading from Hub

9.4 Phase 4: Deployment (Days 9-10)

Task	Duration	Deliverable
Create Gradio interface	2 hours	app.py with image upload and query input
Set up Hugging Face Space	1 hour	Space created and configured
Deploy to Spaces	0.5 hour	Live demo at public URL
Test deployed demo	1 hour	Verify all features working
Add example images	0.5 hour	Gallery of test cases

9.5 Phase 5: Documentation (Days 11-12)

Task	Duration	Deliverable
Write README	2 hours	Comprehensive project documentation
Create architecture diagram	1 hour	System design visualization
Document API	1 hour	API usage guide with examples
Record demo video	1 hour	2-3 minute walkthrough
Prepare resume bullets	0.5 hour	Project description for CV

10. Risk Assessment & Mitigation

Risk	Probability	Impact	Mitigation Strategy
GPU unavailable in Colab	Low	High	Use Colab Pro ($10/month), or switch to Kaggle notebooks
Training doesn't converge	Low	High	Adjust learning rate, increase epochs, verify data quality
Accuracy below 85%	Medium	Medium	Increase training data, adjust hyperparameters, try different LoRA ranks
Inference too slow (>2.5s)	Low	Medium	Use quantization (INT8), optimize batch size, use A100 GPU
Hugging Face Spaces crashes	Low	Medium	Deploy to alternative platform (Railway, Render), optimize memory
Session timeout during training	Medium	Medium	Use Colab Pro (24hr sessions), implement checkpoint resuming
Data format errors	Low	Low	Thorough validation, unit tests for data conversion
Model size too large for deployment	Low	Low	Use LoRA only (256MB vs 14GB full model), implement lazy loading

10.2 Contingency Plans

If Training Fails

- Fallback: Use smaller model (LLaVA-1.5-1.3B) with faster training
- Alternative: Fine-tune only on subset of categories (5 instead of 15)
- Last resort: Use pre-trained CLIP + classification head (simpler architecture)

If Deployment Fails

- Plan B: Deploy as FastAPI backend on Render/Railway + separate frontend
- Plan C: Create local demo with recorded video for portfolio
- Plan D: Package as Docker container for self-hosting

11. Project Timeline & Milestones

Total Duration: 12-14 days (20-25 active hours)**Total Duration: 12-14 days (20-25 active hours)**

Milestone	Completion Date	Success Criteria
M1: Environment Setup	Day 1	Colab notebook with GPU, dataset downloaded
M2: Data Preparation	Day 3	Training/validation JSON files created and validated
M3: Training Complete	Day 6	Model achieves >85% accuracy on validation set
M4: Model Merged & Uploaded	Day 8	Model available on Hugging Face Hub
M5: Demo Deployed	Day 10	Live Gradio demo accessible via public URL
M6: Documentation Complete	Day 12	GitHub repo, README, demo video published
M7: Portfolio Integration	Day 14	Resume updated, LinkedIn showcasing project

11.2 Critical Path

The following tasks are on the critical path and must be completed sequentially:

21. Data preparation (cannot start training without data)
22. Model training (longest duration, cannot parallelize)
23. Model upload to Hub (deployment depends on uploaded model)
24. Gradio Space creation (requires model on Hub)

11.3 Parallel Work Opportunities

- While training: Write documentation, create diagrams, prepare demo video script
- While uploading model: Design Gradio interface, write API documentation
- While testing deployment: Update resume, prepare LinkedIn post

12. Deployment Strategy

12.1 Deployment Options

Option	Cost	Complexity	Recommended For
Hugging Face Spaces (CPU)	Free	Low	Portfolio demo, public showcase ✅
Hugging Face Spaces (GPU)	$0.60/hr	Low	Production with high traffic
Railway/Render	$5-20/month	Medium	Custom backend requirements
AWS SageMaker	$50+/month	High	Enterprise deployment
Local Docker	Free	Medium	Development and testing

12.2 Recommended Deployment (Hugging Face Spaces)

Step-by-Step Deployment

25. Create new Space on Hugging Face with Gradio SDK
26. Create app.py with Gradio interface code
27. Create requirements.txt with dependencies (transformers, torch, peft, gradio)
28. Create README.md with project description and usage
29. Commit files to Space repository (auto-builds in 3-5 minutes)
30. Test deployed demo at https://huggingface.co/spaces/USERNAME/defect-detection
31. Add example images to Space for user convenience
32. Monitor usage via Spaces analytics dashboard

Hardware Configuration

- Initial: CPU Basic (free) - sufficient for demo purposes with ~5-10s inference
- If popular: Upgrade to GPU Persistent ($0.60/hr) - <2s inference
- Memory: 16GB recommended for model loading + inference
- Storage: 20GB for model weights + cache

12.3 Monitoring & Maintenance

- Check Spaces logs daily for errors during first week

- Monitor inference latency and adjust hardware if needed
- Review user feedback via Spaces discussions
- Update model if better checkpoints become available
- Keep dependencies updated for security patches

13. Future Enhancements

13.1 Short-Term Enhancements (1-3 months)

- Multi-image comparison: Upload multiple images, ask 'Which has defects?'
- Confidence visualization: Heatmaps showing model attention on defect regions
- Defect severity scoring: Rate defects as minor/major/critical
- Export functionality: Download analysis results as PDF report
- Batch processing API: Process multiple images in single request

13.2 Medium-Term Enhancements (3-6 months)

- Additional datasets: Expand to other manufacturing domains (PCBs, textiles, automotive)
- Fine-grained defect classification: Identify specific defect types (scratch, dent, crack, etc.)
- Active learning: Allow users to correct predictions, retrain on feedback
- Performance optimization: Quantization to INT8, TensorRT optimization for 10x speedup
- Mobile deployment: Create iOS/Android app with on-device inference

13.3 Long-Term Vision (6-12 months)

- Multi-modal fusion: Combine visual inspection with sensor data (temperature, pressure)
- Temporal analysis: Video input to detect defects in manufacturing process
- Root cause analysis: Identify upstream process issues causing defects
- Predictive maintenance: Predict when equipment will produce defects
- Integration framework: APIs for SAP, MES, ERP systems used in manufacturing

13.4 Research Extensions

- Compare VLM vs traditional CV: Benchmark against YOLO, Faster R-CNN
- Few-shot learning: Adapt to new defect types with <10 examples
- Explainable AI: Generate saliency maps explaining model decisions
- Domain adaptation: Transfer learning to new product categories
- Publish findings: Write technical blog post or conference paper

14. Appendices

14.1 Glossary

Term	Definition
Vision-Language Model (VLM)	AI model that processes both images and text, enabling multimodal understanding
LLaVA	Large Language and Vision Assistant - open-source VLM from Microsoft
LoRA	Low-Rank Adaptation - parameter-efficient fine-tuning method
MVTec AD	MVTec Anomaly Detection dataset - benchmark for industrial defect detection
Zero-shot	Model capability to handle tasks without specific training examples
Inference	Process of using trained model to make predictions on new data
Hugging Face	Platform for sharing and deploying ML models
Gradio	Python library for building ML demo interfaces
CLIP	Contrastive Language-Image Pre-training - vision encoder by OpenAI

14.2 References

- LLaVA Paper: Liu et al. 'Visual Instruction Tuning' (2023)
- LoRA Paper: Hu et al. 'LoRA: Low-Rank Adaptation of Large Language Models' (2021)
- MVTec AD Paper: Bergmann et al. 'MVTec AD: A Comprehensive Dataset for Anomaly Detection' (2019)
- CLIP Paper: Radford et al. 'Learning Transferable Visual Models From Natural Language Supervision' (2021)
- LLaVA GitHub: https://github.com/haotian-liu/LLaVA
- MVTec AD Dataset: https://www.mvtec.com/company/research/datasets/mvtec-ad
- Hugging Face Transformers: https://huggingface.co/docs/transformers

14.3 Technical Stack Summary

Component	Technology	Version
Programming Language	Python	3.10+
Deep Learning Framework	PyTorch	2.2.0
Model Architecture	LLaVA	1.5-7B
Fine-tuning	PEFT (LoRA)	Latest
Vision Encoder	CLIP	ViT-L/14@336
Language Model	Vicuna	7B (LLaMA-based)
Model Loading	Transformers	4.37.2
Training Platform	Google Colab	Pro recommended
Deployment Platform	Hugging Face Spaces	Latest
Web Framework	Gradio	4.16+
Version Control	Git/GitHub	Latest

14.4 Project Links

Upon completion, the following resources will be available:

- GitHub Repository: [To be added]
- Hugging Face Model: [To be added]
- Live Demo (Spaces): [To be added]
- Demo Video: [To be added]
- Technical Blog Post: [To be added]

— END OF DOCUMENT —

This document is a living document and will be updated as the project evolves.